

Real-Time Cryptocurrency Volatility Detection

Project Scoping Brief

Author: Asli Gulcur

Date: November 2, 2025

Project: CMU Heinz - Operationalizing AI Assignment

1. Use Case: What Problem Are We Solving?

Business Context: Cryptocurrency markets are extremely volatile, with prices changing rapidly within seconds. Traders, investors, and automated trading systems need real-time alerts about sudden price volatility to make informed decisions about buying, selling, or adjusting positions.

The Problem:

- Bitcoin and other cryptocurrencies can experience price swings of 5-10% within minutes
- Current solutions rely on delayed data or manual monitoring
- Traders miss opportunities or incur losses due to delayed volatility detection
- No real-time system to predict short-term (60-second) volatility spikes

Who This Helps:

- Day traders making rapid trading decisions
- Automated trading bots needing volatility signals
- Risk management teams monitoring exposure
- Portfolio managers adjusting hedging strategies

Value Proposition: A real-time system that predicts volatility 60 seconds ahead, giving users time to:

- Execute trades before major price movements
 - Adjust stop-loss orders to protect positions
 - Reduce risk exposure during volatile periods
 - Capitalize on arbitrage opportunities
-

2. Prediction Goal: What Exactly Are We Predicting?

Primary Objective: Predict whether Bitcoin (BTC-USD) will experience HIGH or LOW volatility in the next 60 seconds.

Definition of Volatility:

- **High Volatility:** Price standard deviation $>$ [threshold] over 60-second window
- **Low Volatility:** Price standard deviation $\leq$ [threshold] over 60-second window

Prediction Window:

- **Input:** Last 5 minutes of tick data (price, volume, bid-ask spread)

- **Output:** Binary classification (HIGH/LOW) for next 60 seconds
- **Update Frequency:** Every 10 seconds (rolling predictions)

Features We'll Use:

1. Price-based:

- Rolling mean/std of prices (30s, 60s, 120s windows)
- Price change velocity (rate of change)
- High-low price range over recent windows

2. Volume-based:

- 24-hour trading volume trends
- Volume spikes (sudden increases)
- Volume-price correlation

3. Market microstructure:

- Bid-ask spread (market liquidity indicator)
- Trade frequency (number of trades per second)
- Buy vs sell pressure

Example: At 3:00:00 PM, using data from 2:55:00-3:00:00, predict if volatility at 3:01:00 will be HIGH or LOW.

3. Success Metrics: How Do We Measure Performance?

Primary Metric: Accuracy

- Target: $\geq 70\%$ classification accuracy
- Rationale: Random guessing = 50%, industry standard for trading signals = 60-70%

Secondary Metrics:

1. Precision (for HIGH volatility class)

- Target: $\geq 75\%$
- Why: False alarms are costly (unnecessary trades, fees)
- When we predict HIGH, we want to be right 75% of the time

2. Recall (for HIGH volatility class)

- Target: $\geq 65\%$
- Why: Missing volatile periods means missed opportunities
- We want to catch at least 65% of actual high volatility events

3. F1-Score

- Target: ≥ 0.70
- Why: Balance between precision and recall

4. Prediction Latency

- Target: **< 100ms** from data arrival to prediction
- Why: Real-time systems need fast responses
- Measured: Time from Kafka message receipt to model output

Business Impact Metrics:

- **Alert Timeliness:** Predictions delivered 60 seconds before volatility event
- **System Uptime:** $\geq 99\%$ availability during trading hours
- **Data Freshness:** ≤ 1 second delay from Coinbase to model input

Evaluation Approach:

- Train on 7 days of historical data
 - Test on next 24 hours (hold-out set)
 - Monitor performance drift over time with Evidently
-

4. Risk Assumptions: What Could Go Wrong?

Technical Risks

1. Data Availability

- **Risk:** Coinbase WebSocket disconnects or lags
- **Mitigation:** Implemented reconnection logic with 5 retry attempts
- **Impact if occurs:** Missing data → degraded predictions
- **Monitoring:** Track connection uptime, message gaps

2. Kafka Performance

- **Risk:** Kafka cannot handle message volume (>100 msgs/sec)
- **Mitigation:** Kafka designed for high throughput; monitoring lag
- **Impact if occurs:** Backpressure → prediction delays
- **Monitoring:** Consumer lag metrics in Kafka

3. Model Latency

- **Risk:** Predictions take >100 ms, defeating "real-time" purpose
- **Mitigation:** Use lightweight model (e.g., Random Forest, not deep learning)
- **Impact if occurs:** Predictions arrive too late to be useful
- **Monitoring:** Track inference time with MLflow

Data Quality Risks

4. Concept Drift

- **Risk:** Market patterns change; model becomes stale
- **Mitigation:** Evidently monitoring for data/prediction drift
- **Impact if occurs:** Accuracy degrades over weeks/months
- **Monitoring:** Weekly drift reports

5. Insufficient Training Data

- **Risk:** 7 days may not capture all market conditions
- **Assumption:** Recent data more relevant than old data for crypto
- **Impact if occurs:** Model overfits to recent patterns
- **Mitigation:** Cross-validation, monitor test performance

Business/Market Risks

6. Market Regime Changes

- **Risk:** Bull/bear market transitions break patterns
- **Assumption:** Volatility patterns are somewhat stable
- **Impact if occurs:** Sudden accuracy drop during market crashes
- **Mitigation:** Retrain model weekly; alert on performance drops

7. Low Volatility Periods

- **Risk:** During stable markets, class imbalance (mostly LOW volatility)
- **Assumption:** Need balanced training data
- **Impact if occurs:** Model predicts LOW for everything
- **Mitigation:** Class balancing techniques (SMOTE, class weights)

8. Regulatory Changes

- **Risk:** Crypto regulations could affect market structure
- **Assumption:** Coinbase API remains accessible and free
- **Impact if occurs:** Need to switch data source
- **Mitigation:** Design system to be data-source agnostic

Operational Risks

9. System Failures

- **Risk:** Docker container crashes, service downtime
- **Mitigation:** Docker restart policies, health checks
- **Impact if occurs:** Missed trading opportunities
- **Monitoring:** Uptime alerts, automated restarts

10. Resource Constraints

- **Risk:** Running out of disk space (Kafka logs grow large)
- **Assumption:** Adequate storage provisioned
- **Impact if occurs:** System halts
- **Mitigation:** Log rotation, storage monitoring

5. Project Scope & Timeline

In Scope:

- Real-time data ingestion from Coinbase (BTC-USD)

- Kafka streaming infrastructure
- Feature engineering pipeline
- Binary volatility classification model
- MLflow experiment tracking
- Evidently drift monitoring
- Dockerized deployment

Out of Scope (for this project):

- Multi-currency predictions (only BTC-USD)
- Regression (exact volatility values)
- Trading execution/backtesting
- Production deployment to cloud
- Multi-model ensemble

Timeline:

-  Milestone 1: Streaming setup (Week 1) - 64% complete
-  Milestone 2: Feature engineering, EDA (Week 2)
-  Milestone 3: Modeling, evaluation (Week 3)

6. Success Criteria Summary

System is successful if:

1. Achieves $\geq 70\%$ accuracy on hold-out test set
2. Predictions delivered with $< 100\text{ms}$ latency
3. System runs continuously for 24+ hours without crashes
4. Drift monitoring detects and alerts on performance degradation
5. All code containerized and reproducible

Deliverables:

- Working streaming pipeline (Kafka + WebSocket)
- Trained ML model with documented performance
- Drift monitoring dashboard
- Docker containers for all components
- Documentation and reproducible setup

Notes & Assumptions

- **Data Source:** Coinbase Pro WebSocket (free, no authentication required for market data)
 - **Infrastructure:** Local Docker environment (sufficient for prototype)
 - **Model Complexity:** Start simple (Random Forest), iterate if needed
 - **Retraining Strategy:** Manual retraining weekly (not automated in this scope)
 - **Target Users:** Educational/demonstration (not production trading system)
-

Prepared by: [Your Name]
Version: 1.0
Last Updated: November 2, 2025